

1. Data Preprocessing

The dataset consists five clothing classes (T-Shirts, Pants, Socks, Shorts, and Long-sleeved), required standardization for stable training. All images were confirmed to be standard RGB. To address size inconsistency while preserving content: images were first resized to 256×256, then a random 224×224 crop was taken during training. Pixel values were converted to tensors and normalized to accelerate gradient convergence. The dataset was divided using to maintain class balance: 80% training, 10% validation, and 10% test.

2. CNN Architecture and Model Variants

The model is built using CNN architecture designed to extract layered visual details from the clothing images.

The network shrinks the spatial dimensions of the image by half five times (from 224x224 down to 7x7) using MaxPool2d, while the channel depth increases: 3 → 16 → 32 → 64 → 128 → 128 to extract increasingly complex features. After flattening (resulting in $128 * 7 * 7 = 6272$ features), the classifier uses a single hidden Linear layer with 88 neurons. The total trainable parameter count is 798,229.

- **Adaptive Learning (ReduceLROnPlateau):** Instead of sticking to one learning rate, we added a scheduler. If the validation loss stalled for 2 epochs (patience=2), the learning rate was cut in half (factor=0.5). This allows the model to take big steps early on, and tiny precise steps when it's getting close to the optimal solution.
- **Regularization:** We combined a dropout layer (p=0.3) in the classifier in the Adam optimizer, and Batch Normalization. Because the data was augmented and the learning rate was adaptive, these regularizers worked together.
- **Hyperparameters:** Adam optimizer (LR = 0.001), Batch Size = 64, Epochs = 30.

Some Variants:

Batch Normalization Model-Old Data:

(LEARNING_RATE = 0.002, EPOCHS = 30, BATCH_SIZE = 64)

Why: As weights update during backpropagation, the distribution of layer inputs constantly changes. BN standardizes these inputs across each mini-batch (setting a mean of 0 and variance of 1) so deeper layers don't have to continuously adapt to changing distributions. Prevents vanishing gradients, allowing the network to use a higher learning rate, converge much faster, and smooth out the optimization path.

What do we got:

- By Step 28, train_accuracy reached 73.3%, while your val_accuracy hovered around 63% efficiently.
- Clear feature extraction across categories: Socks achieved 0.80 F1-Score (63 correct) due to highly distinct normalized shapes, while Pants reached 0.73 F1-Score (52 correct).

Regularized Model (With L2):

(LEARNING_RATE = 0.002, EPOCHS = 30, BATCH_SIZE = 64)

Why: It adds a penalty proportional to the square of the magnitude of the weights to the loss function. This penalizes excessively large weights, forcing the network to keep the weight values small and distributed.

What do we got:

- Because nn.BatchNorm2d already inherently acts as a regularizer via mini-batch noise, stacking L2 regularization on top of it introduced too much constraint. Instead of reducing overfitting, it choked the network's capacity, making the model underfit.
- The L2 penalty was strict, preventing the network from learning fine details. Train accuracy peaked lower at ~67% , and validation accuracy dropped to ~58%.
- F1-scores drop across all categories.

W&B Sweep

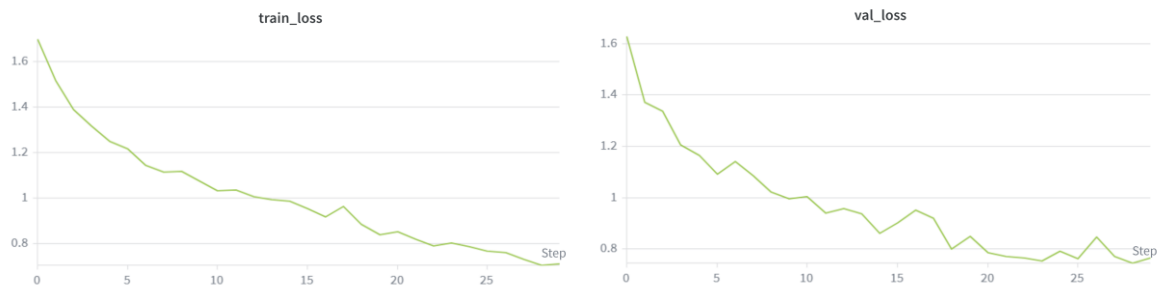
(magic-sweep-4, icy-sweep-3, restful-sweep-2, hardy-sweep-1)

Why: To systematically explore the hyperparameter space (such as varying learning rates and optimization paths) using automated tracking to locate the most stable and efficient convergence trajectory without manual trial and error.

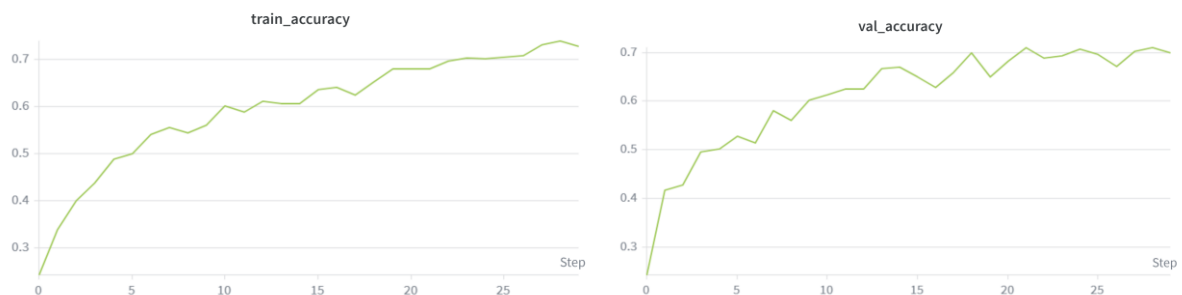
What we got:

- All 4 versions showed similar results, with both train_loss and val_loss going down smoothly together. This proves the model structure is highly stable even when we change the settings.
- The sweep did not trigger early stopping too soon, which allowed the training and validation accuracies to grow steadily without separating.
- Validation accuracy grew well during the 15 epochs, reaching between 55% and 58% for all runs.

3. Training Curves and Evaluation Metrics

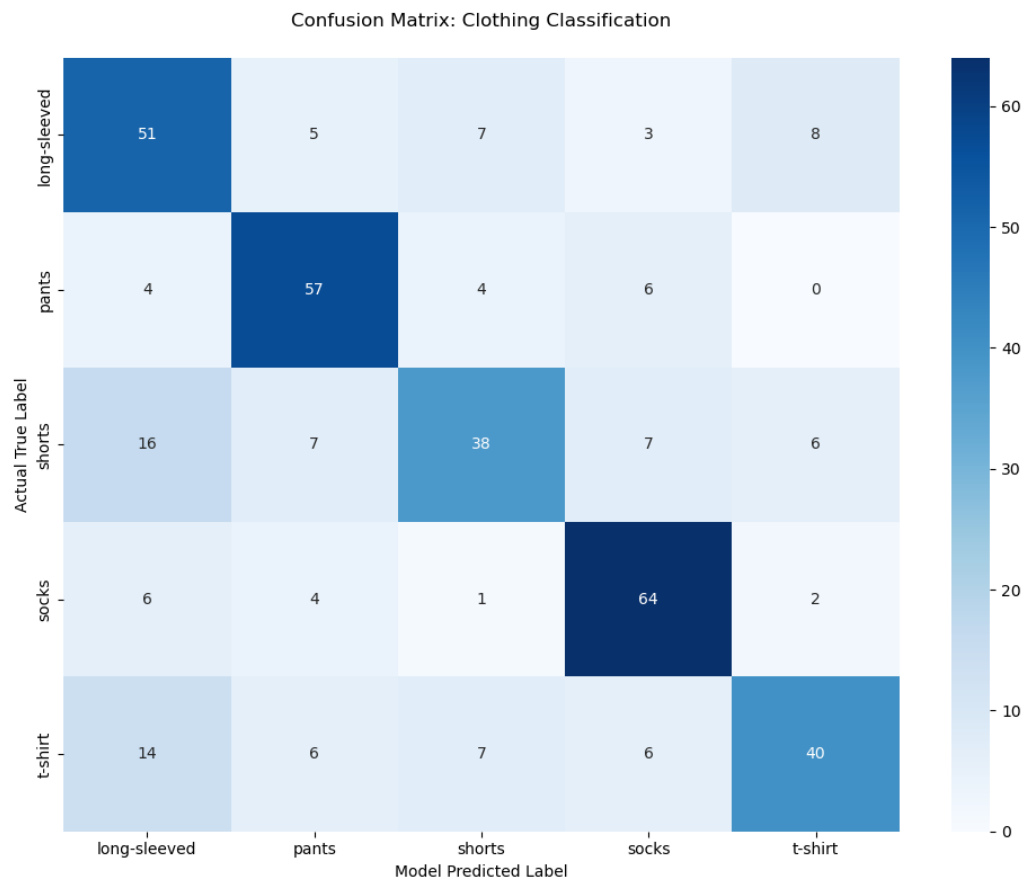


Both `train_loss` and `val_loss` go down smoothly stabilizing around ~ 0.75 . The validation loss doesn't go back up. This shows that data augmentation and dropout worked well. They stopped the network from just memorizing the training images. Instead, the model successfully learned real patterns that work on new unseen pictures.



Training accuracy reached around 74%, and validation accuracy stopped at about 70%. This small difference shows that the model is good at working with new, unseen images. It proves that the network actually learned general clothing shapes (like sleeves) instead of just memorizing the training pictures.

Confusion Matrix & Class Metrics



Class	Precision	Recall	F1-Score	Support
long-sleeved	0.5604	0.6892	0.6182	74
pants	0.7215	0.8028	0.7600	71
shorts	0.6667	0.5135	0.5802	74
socks	0.7442	0.8312	0.7853	77
t-shirt	0.7143	0.5479	0.6202	73

The confusion matrix and metrics show balanced learning across the board:

- Socks remain the easiest category, dominating with an 0.78 F1-Score (64 correctly predicted).
- Pants reached 0.76 F1-Score (57 correctly predicted).

- T-shirts and Shorts (the hardest categories to separate due to similar fabrics and borders) both saw improvements, achieving F1-Scores around 0.62 and 0.58.

4. Misclassified Images Analysis

Index 12



Truth: pants | Gussed: shorts



Truth: shorts | Gussed: long-sleeved



Truth: long-sleeved | Gussed: pants



Truth: t-shirt | Gussed: long-sleeved



Truth: socks | Guessed: t-shirt



Truth: shorts | Guessed: socks



Truth: t-shirt | Guessed: long-sleeved



Truth: socks | Guessed: pants

Shorts as Long-Sleeved / Socks as T-Shirts: When the model crops a section from the center fabric of a sock or pair of shorts, it misses important parts like leg openings or heels, so it gets confused by the flat cloth texture.

T-Shirts as Long-Sleeved: Big, loose t-shirts or folded short sleeves can look like long sleeves, with wrinkly fabric and bad camera angles.

Long-Sleeved as Pants: When a long-sleeved shirt is laid flat, the long sleeves look just like the two long legs of a pair of pants, which tricks the model.